

LAPORAN PRAKTIKUM PEKAN 4
PEMOGRAMAN BERORIENTASI OBJEK

Ekspansi Produk Keuangan (Inheritance)



Oleh:
Naysha Aprilia Rizki (2511531014)
Kelas B

Dosen Pengampu:
Nurfiah, S.ST., M.Kom.

Asisten Praktikum:
Ezza Addini

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
PADANG
2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan sistem perbankan, terdapat berbagai jenis rekening yang memiliki karakteristik dan fitur yang berbeda. Contohnya adalah rekening tabungan yang memiliki fitur suku bunga, rekening giro yang memiliki fasilitas overdraft, serta rekening VIP yang dapat memiliki fitur tambahan tertentu. Jika setiap jenis rekening dibuat sebagai class yang berdiri sendiri, akan terdapat banyak kode yang sama dan dapat menyebabkan pengulangan kode.

Untuk mengatasi hal tersebut, digunakan konsep Inheritance (pewarisan) dalam Pemrograman Berorientasi Objek. Inheritance memungkinkan sebuah class baru mewarisi atribut dan method dari class yang sudah ada. Pada praktikum ini, class "Rekening" digunakan sebagai superclass, sedangkan "RekeningTabungan", "RekeningGiro", dan "RekeningVIP" dapat digunakan sebagai subclass yang memiliki fitur tambahan sesuai jenis rekeningnya.

Penerapan inheritance dilakukan menggunakan keyword "extends", sedangkan "super()" digunakan untuk memanggil constructor dari class induk. Selain itu, praktikum ini juga menerapkan konsep upcasting dan downcasting untuk mengelola berbagai jenis objek rekening dalam satu koleksi.

1.2 Tujuan

1. Memahami konsep Inheritance (pewarisan) dalam Pemrograman Berorientasi Objek.
2. Memahami perbedaan antara superclass dan subclass.
3. Menerapkan keyword "extends" untuk membuat class turunan dari class "Rekening".
4. Menerapkan "super()" untuk memanggil constructor dari class induk.

5. Membuat berbagai jenis rekening dengan fitur tambahan melalui inheritance.
6. Memahami dan menerapkan konsep upcasting dan downcasting pada objek rekening.
7. Mengembangkan program perbankan agar dapat menangani beberapa jenis produk rekening.

1.3 Manfaat

1. Mempermudah pembuatan class baru karena dapat menggunakan kembali atribut dan method dari superclass.
2. Mengurangi pengulangan kode sehingga program menjadi lebih terstruktur dan mudah dikembangkan.
3. Membantu memahami penerapan konsep inheritance dalam kasus yang lebih nyata, yaitu sistem perbankan.
4. Memahami cara membuat subclass dengan karakteristik dan fitur tambahan yang berbeda.
5. Memahami penggunaan upcasting dan downcasting dalam pengelolaan objek.
6. Meningkatkan kemampuan dalam merancang program Java yang menerapkan prinsip code reusability.

BAB II

PEMBAHASAN

2.1 Teori

Seiring berkembangnya sistem perbankan, bank tidak hanya memiliki satu jenis rekening. Ada "Rekening Tabungan" yang mana akan mendapat bunga bulanan dan "Rekening Giro" yang memiliki fasilitas pinjaman atau overdraft. Jika kita membuat class baru dari nol untuk masing masing entitas tersebut, kita akan melanggar prinsip DRY (Don't Repeat Yourself) karena menduplikasi banyak kode logika PIN dan mutasi. "Is-A" contohnya Tabungan is-a Rekening. langsung ke kelas induk:

- Konsep Pewarisan (Inheritance): Menurut Deitel dan Deitel (2017), pewarisan adalah bentuk software reuse di mana class baru diciptakan dengan menggunakan atribut dan behavior dari class yang sudah ada, lalu menambahkan kapabilitas baru. Hubungan ini dikenal sebagai relasi

- Penggunaan Keyword extends dan super : Di Java, subclass mewarisi kelas induk menggunakan kata kunci extends. Sementara itu, kata kunci super berfungsi sebagai jembatan

- super() digunakan di dalam konstruktor kelas anak untuk mengeksekusi inisialisasi konstruktor induk. Pemanggilan ini wajib berada di baris paling pertama di dalam konstruktor anak (Oracle, 2023).
- super.namaMethod() digunakan jika kelas anak perlu memanggil metode asli milik induk yang mungkin telah dimodifikasi (di-override).

- Pentingnya protected: Agar kelas anak (Tabungan, Giro) bisa memanipulasi saldo dan mencatat riwayatTransaksi tanpa harus membuka akses tersebut ke publik, atribut tersebut harus menggunakan modifier protected.

- Upcasting dan Downcasting: Karena adanya relasi Is-A, Java mengizinkan konversi tipe referensi antar hierarki kelas (Eckel, 2006):

- Upcasting (Implisit): Menyimpan objek kelas anak ke dalam variabel referensi kelas induk contohnya `Rekening akun = new RekeningTabungan(...)`. Proses ini otomatis dan aman. Fungsinya sangat krusial agar kita bisa menyimpan berbagai jenis produk rekening ke dalam satu koleksi yang seragam seperti `ArrayList` tipe `Rekening`.

- Downcasting (Eksplisit): Memaksa referensi kelas induk kembali menjadi kelas anak spesifik contoh `RekeningTabungan tab = (RekeningTabungan) akun;`. Proses ini rawan memicu error `ClassCastException` saat program berjalan jika objek aslinya ternyata bukan `Tabungan`. Oleh karena itu, downcasting wajib didahului oleh pengecekan tipe menggunakan operator `instanceof`.

2.2 Langkah Langkah

1. Membuat Class Transaksi

- 1) Pertama saya membuat Package “`prak4_Inheritance`”
- 2) Selanjutnya dilanjutkan dengan membuat Class `Transaksi`, Berikut Screenshot lengkap nya

```
1 package prak4_Inheritance;
2
3 public class Transaksi {
4     //mengubah atribut id private
5     private String idTransaksi;
6     private String jenis;
7     private double nominal;
8
9     public Transaksi(String id, String jenis, double nominal) {
10         this.idTransaksi = id;
11         this.jenis = jenis;
12         this.nominal = nominal;
13     }
14     // Hanya nyediakan getter (Read Only)
15     public String getIdTransaksi() { return idTransaksi;}
16     public String getJenis() { return jenis; }
17     public double getNominal() { return nominal;}
18
19     public void cetakDetail() {
20         System.out.println("ID: " + idTransaksi + " | Jenis:"
21             + jenis + " | Nominal: Rp" + nominal);
22     }
23 }
24 }
```

Penjelasan:

1) Buat class Transaksi

```
public class Transaksi {
```

Ini mengartikan kita sedang membuat sebuah class bernama Transaksi. Class bisa dianggap sebagai cetakan/model untuk membuat objek Transaksi.

2) Membuat State/Atribut

```
//mengubah atribut id private  
private String idTransaksi;  
private String jenis;  
private double nominal;
```

Atribut ini merupakan variable yang menyimpan informasi tentang suatu objek. Pada class transaksi ini tersimpan data berupa

- idTransaksi dengan tipe data string
- jenis dengan tipe data string
- nominal dengan tipe data double.

Kata private digunakan agar atribut hanya dapat diakses secara langsung dari dalam class Transaksi. Hal ini merupakan penerapan konsep **enkapsulasi**.

3) Buat Constructor

```
public Transaksi(String id, String jenis, double nominal) {  
    this.idTransaksi = id;  
    this.jenis = jenis;  
    this.nominal = nominal;  
}
```

Constructor ini digunakan untuk menginisialisasikan atau memberi nilai awal pada objek Transaksi Ketika objek dibuat.

- idTransaksi di inisialisasikan dengan nilai id
- jenis diinisialisasikan dengan nilai jenis transaksi, yaitu jenis transaksi debit atau kredit
- nominal diinisialisasikan dengan nilai nominak transaksi

this digunakan untuk menunjukkan atribut yang berada di dalam class Transaksi.

4) Membuat Method Getter

```
// Hanya nyediakan getter (Read Only)
public String getIdTransaksi() { return idTransaksi;}
public String getJenis() { return jenis; }
public double getNominal() { return nominal;}
```

Method getter digunakan untuk mengambil atau membaca nilai atribut yang bersifat private. Pada class Transaksi terdapat tiga getter, yaitu:

- getIdTransaksi() digunakan untuk mengambil nilai idTransaksi.
- getJenis() digunakan untuk mengambil nilai jenis.
- getNominal() digunakan untuk mengambil nilai nominal.

Getter digunakan karena atribut dibuat private, sehingga nilai atribut tidak dapat diakses secara langsung dari luar class

5) Buat Method cetakDetail

```
public void cetakDetail() {
    System.out.println("ID: " + idTransaksi + " | Jenis:"
        + jenis + " | Nominal: Rp" + nominal);
}
```

Method cetakDetail() digunakan untuk menampilkan detail transaksi yang berisi ID transaksi, jenis transaksi, dan nominal transaksi. Program akan menampilkan informasi dengan format “ID: (ID transaksi) | Jenis: (jenis transaksi) | Nominal: Rp (nominal)”.

2. Membuat Class Rekening

Salin Code Program yang sudah dibuat pada praktikum prak2_Relasi, Dan ubah atau tambah beberapa bagian saja. Berikut Screenshot

```

1 package prak4_Inheritance;
2 import java.util.ArrayList;
3
4 public class Rekening {
5     //Batas atas dari class Rekening
6     private String nomorRekening;
7     private String namaPemilik;
8     private String pin;
9
10    //Gunakan protect agar subclass bisa mengakses langsung
11    protected double saldo;
12    protected ArrayList<Transaksi> riwayatTransaksi;
13
14    //Isi kontruktor dan Method lain sama dengan modul 3

```

Penjelasan:

```

public class Rekening {
    //Batas atas dari class Rekening
    private String nomorRekening;
    private String namaPemilik;
    private String pin;

```

Tiga atribut tersebut menggunakan private, sehingga hanya dapat diakses langsung dari dalam class Rekening.

- nomorRekening → menyimpan nomor rekening.
- namaPemilik → menyimpan nama pemilik rekening.
- pin → menyimpan PIN rekening.

```

//Gunakan protect agar subclass bisa mengakses langsung
protected double saldo;
protected ArrayList<Transaksi> riwayatTransaksi;

```

Kedua atribut menggunakan protected, sehingga dapat diakses oleh class Rekening dan subclass-nya.

- saldo → menyimpan jumlah saldo rekening dalam tipe double.
- riwayatTransaksi → menyimpan daftar riwayat transaksi menggunakan ArrayList dengan objek Transaksi.

3. Membuat Class RekeningTabungan

```
1 package prak4_Inheritance;
2
3 public class RekeningTabungan extends Rekening {
4     //Atribut spesifik yang hanya dimiliki oleh Tabungan
5     private double sukuBunga;
6
7     //Constructor Subclass
8     public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal,
9         double sukuBunga) {
10         //super() memanggil constructor kelas induk (Rekening) WAJIB berada di baris pertama
11         super(nomor, nama, saldoAwal, pinAwal);
12         this.sukuBunga = sukuBunga;
13     }
14
15     public void tambahBungaAkhirBulan() {
16         //Menghitung bunga
17         //Mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan ?
18         double nominalBunga = saldo * (sukuBunga / 100);
19         saldo += nominalBunga;
20
21         //Mencatat riwayat transaksi
22         String idTrx = "TRX-B-" + System.currentTimeMillis();
23         riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));
24
25         System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp"
26             + nominalBunga);
27     }
28 }
29
```

Penjelasan

```
public class RekeningTabungan extends Rekening {
```

Membuat RekeningTabungan sebagai subclass dari Rekening, sehingga dapat mewarisi atribut dan method dari class induk.

```
//Atribut spesifik yang hanya dimiliki oleh Tabungan
private double sukuBunga;
```

Menyimpan nilai suku bunga khusus untuk rekening tabungan.

```
//Constructor Subclass
public RekeningTabungan(String nomor, String nama, double saldoAwal, String pinAwal,
    double sukuBunga) {
    //super() memanggil constructor kelas induk (Rekening) WAJIB berada di baris pertama
    super(nomor, nama, saldoAwal, pinAwal);
    this.sukuBunga = sukuBunga;
}
```

Merupakan constructor untuk mengisi data rekening dan suku bunga. `super()` digunakan untuk memanggil constructor class Rekening dan harus berada di baris pertama, sedangkan `this.sukuBunga` mengisi atribut suku bunga.

```
public void tambahBungaAkhirBulan() {
    //Menghitung bunga
    //Mengapa bisa mengakses saldo secara langsung dari class RekeningTabungan ?
    double nominalBunga = saldo * (sukuBunga / 100);
    saldo += nominalBunga;
```

Menghitung nominal bunga berdasarkan saldo dan suku bunga, kemudian menambahkan bunga tersebut ke saldo. saldo dapat diakses langsung karena menggunakan `protected` pada class Rekening.

```
//Mencatat riwayat transaksi
String idTrx = "TRX-B-" + System.currentTimeMillis();
riwayatTransaksi.add(new Transaksi(idTrx, "Bunga", nominalBunga));
```

Membuat ID transaksi berdasarkan waktu dan mencatat transaksi bunga ke dalam riwayatTransaksi.

```
System.out.println("Bunga " + sukuBunga + "% berhasil ditambahkan: Rp"
+ nominalBunga);

}
}
```

Menampilkan informasi bahwa bunga berhasil ditambahkan beserta nominalnya.

4. Membuat Class RekeningGiro

```
1 package prak4_Inheritance;
2
3 public class RekeningGiro extends Rekening {
4     private double batasOverdraft;
5
6     public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal,
7         double batasOverdraft) {
8         //memanggil inisialisasi dasar superclass
9         super(nomor, nama, saldoAwal, pinAwal);
10        this.batasOverdraft = batasOverdraft;
11    }
12
13    //Getter khusus Giro
14    public double getBatasOverdraft() {
15        return batasOverdraft;
16    }
17    // (Catatan: Rencana harga limit overdraft akan diselesaikan dimodul 5)
18 }
```

Penjelasan

```
public class RekeningGiro extends Rekening {
    private double batasOverdraft;
```

Membuat RekeningGiro sebagai subclass dari Rekening, sehingga mewarisi atribut dan method dari class induk.

```
private double batasOverdraft;
```

Menyimpan batas overdraft, yaitu batas saldo negatif yang dapat digunakan pada rekening giro.

```
public RekeningGiro(String nomor, String nama, double saldoAwal, String pinAwal,
    double batasOverdraft) {
    //memanggil inisialisasi dasar superclass
    super(nomor, nama, saldoAwal, pinAwal);
    this.batasOverdraft = batasOverdraft;
}
```

Constructor digunakan untuk mengisi data dasar rekening melalui super() dan mengisi batasOverdraft dengan nilai yang diberikan.

```

public double getBatasOverdraft() {
    return batasOverdraft;
}

```

Merupakan getter yang digunakan untuk mengambil nilai batas overdraft karena atribut batasOverdraft bersifat private.

TUGAS:

Perbarui Main.java agar mendukung :

1. Pembuatan Multi-Produk

Modifikasi case 1: Buka Rekening Baru di Main.java.

- 1) Setelah user menginputkan Saldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis"
- 2) Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.
- 3) Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.
- 4) Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

```

case 1:
    System.out.print("Masukan No Rekening: ");
    String no = input.nextline();
    System.out.print("Masukan Nama Pemilik: ");
    String nama = input.nextline();
    System.out.print("Masukan Saldo Awal: ");
    double saldo = input.nextDouble();
    input.nextline();

    // minta input PIN
    System.out.print("Masukan PIN(6 digit):");
    String pin = input.nextline();

    //Pilih Produk
    System.out.println("\nPilih Produk:");
    System.out.println("1.Tabungan Umum");
    System.out.println("2.Giro Bisnis");
    System.out.print("Pilih Produk: ");
    int produk = input.nextInt();
    input.nextline();
    Rekening rekeningBaru = null;
    if (produk == 1) {
        System.out.print("Masukan Suku Bunga (%): ");
        double sukuBunga = input.nextDouble();
        input.nextline();

        rekeningBaru = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
        System.out.println("Rekening Tabungan Berhasil dibuat.");
    } else if (produk == 2) {
        System.out.print("Masukan Batas Overdraft: ");
        double batasOverdraft = input.nextDouble();
        input.nextline();

        rekeningBaru = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);
        System.out.println("Rekening Giro Berhasil dibuat.");
    } else {
        System.out.println("Pilihan produk tidak valid.");
    }

    break;
}

daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;

break;

```

Penjelasan:

```
case 1:
    System.out.print("Masukan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukan Saldo Awal: ");
    double saldo = input.nextDouble();
    input.nextLine();
```

Meminta input nomor rekening, nama pemilik, dan saldo awal. `nextLine()` digunakan untuk membaca teks, sedangkan `nextDouble()` untuk membaca angka.

```
// minta input PIN
System.out.print("Masukan PIN(6 digit):");
String pin = input.nextLine();
```

Meminta pengguna memasukkan PIN rekening.

```
//Opsi Produk
System.out.println("\nPilih Produk:");
System.out.println("1.Tabungan Umum");
System.out.println("2.Giro Bisnis");
System.out.print("Pilih Produk: ");
int produk = input.nextInt();
input.nextLine();
Rekening rekeningBaru = null;
```

Menampilkan pilihan jenis rekening, yaitu Tabungan Umum atau Giro Bisnis, kemudian menyimpan pilihan pengguna ke variabel `produk`. Membuat variabel `rekeningBaru` bertipe `Rekening` yang awalnya bernilai `null`. Variabel ini nantinya dapat menyimpan objek `RekeningTabungan` atau `RekeningGiro`.

```
Rekening rekeningBaru = null;
if (produk == 1) {
    System.out.print("Masukan Suku Bunga (%): ");
    double sukuBunga = input.nextDouble();
    input.nextLine();

    rekeningBaru = new RekeningTabungan(no, nama, saldo, pin, sukuBunga);
    System.out.println("Rekening Tabungan Berhasil dibuat.");
}
```

Jika pengguna memilih Tabungan, program meminta suku bunga lalu membuat objek `RekeningTabungan` dan menyimpannya ke `rekeningBaru`.

```

        System.out.println("Rekening Tabungan Berhasil dibuat.");
    } else if (produk == 2) {
        System.out.println("Masukan Batas Overdraft: ");
        double batasOverdraft = input.nextDouble();
        input.nextLine();

        rekeningBaru = new RekeningGiro(no, nama, saldo, pin, batasOverdraft);
        System.out.println("Rekening Giro Berhasil dibuat.");
    }
}

```

Jika memilih Giro, program meminta batas overdraft lalu membuat objek RekeningGiro dan menyimpannya ke rekeningBaru.

```

        System.out.println("Rekening Giro Berhasil dibuat.");
    } else {
        System.out.println("Pilihan produk tidak valid.");

        break;
    }

    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
}

```

Jika pilihan selain 1 atau 2, program menampilkan pesan bahwa pilihan tidak valid dan menghentikan proses case.

OUTPUT

```

Main (5) [Java Application] C:\Program Files\Java\jdk-24\bin\javaw.exe (5)
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
7. Simulasi Akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu:
1
Masukan No Rekening: 001
Masukan Nama Pemilik: nay
Masukan Saldo Awal: 200000
Masukan PIN(6 digit): 123456

Pilih Produk:
1. Tabungan Umum
2. Giro Bisnis
Pilih Produk: 1
Masukan Suku Bunga (%): 2
Rekening atas nama nay berhasil dibuat.
Rekening Tabungan Berhasil dibuat.

```

```

=== SISTEM PERBANKAN MINI ===

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
0.Keluar
Pilih Menu:
1
Masukan No Rekening: 001
Masukan Nama Pemilik: nay
Masukan Saldo Awal: 200000
Masukan PIN(6 digit):123456

Pilih Produk:
1.Tabungan Umum
2.Giro Bisnis
Pilih Produk: 2
Masukan Batas Overdraft:
100000
Rekening atas nama nay berhasil dibuat.
Rekening Giro Berhasil dibuat.

```

2. Simulasi Waktu Akhir Bulan

- 1) Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.
- 2) Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof.
- 3) Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().
- 4) Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

```

case 7:
    if(akunAktif == null) {
        System.out.println("Error : Anda belum membuka rekening!");
    } else if (akunAktif instanceof RekeningTabungan) {
        // DOWNCASTING
        RekeningTabungan tabungan =
            (RekeningTabungan) akunAktif;

        tabungan.tambahBungaAkhirBulan();
    } else {
        System.out.println(
            "Gagal: Fitur bunga akhir bulan "
            + "hanya berlaku untuk Rekening Tabungan."
        );
    }

    break;

```

Penjelasan

```
case 7:
    if(akunAktif == null) {
        System.out.println("Error : Anda belum membuka rekening!");
    }
}
```

Mengecek apakah pengguna sudah memiliki rekening aktif. Jika belum, program menampilkan pesan error.

```
System.out.println("Error : Anda belum membuka rekening!");
} else if (akunAktif instanceof RekeningTabungan) {

    // DOWNCASTING
    RekeningTabungan tabungan =
        (RekeningTabungan) akunAktif;

    tabungan.tambahBungaAkhirBulan();
}
```

Mengecek apakah rekening aktif merupakan RekeningTabungan menggunakan instanceof. Jika iya, dilakukan downcasting dari tipe Rekening menjadi RekeningTabungan, kemudian method tambahBungaAkhirBulan() dijalankan untuk menambahkan bunga.

```
} else {

    System.out.println(
        "Gagal: Fitur bunga akhir bulan "
        + "hanya berlaku untuk Rekening Tabungan."
    );
}

break;
```

Jika rekening aktif bukan RekeningTabungan, fitur bunga tidak dapat digunakan karena hanya berlaku untuk rekening tabungan.

OUTPUT

```
Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
7.Simulasi Akhir Bulan (Khusus Tabungan)
0.Keluar
Pilih Menu:
7
Bunga 2.0% berhasil ditambahkan: Rp4000.0
```